High-Level Design (HLD)
Credit Risk Prediction Platform

1 Introduction

This document describes the high-level design of the Credit Risk Prediction Platform. The system is designed to predict loan default probability using machine learning and provide an interactive interface for non-technical users.

The architecture follows a modular and loosely coupled design to ensure scalability, maintainability, and ease of integration.

2 System Overview

The system consists of the following major components:

- Frontend UI (Streamlit)
- Backend API (FastAPI)
- Machine Learning Model (LightGBM)
- Data Processing Pipeline (DVC)
- Workflow Orchestration (Airflow)
- Experiment Tracking (MLflow)
- Monitoring (Prometheus & Grafana)

3 Design Principles

3.1 Loose Coupling

The frontend and backend are independent systems communicating via REST APIs. This ensures flexibility and allows independent scaling and deployment.

3.2 Modularity

Each component (data pipeline, training, inference, monitoring) is implemented as a separate module.

3.3 Scalability

The architecture allows scaling of API services and data pipelines independently.

3.4 Reproducibility

DVC ensures reproducibility of data pipelines and model training.

4 System Components

4.1 Frontend (Streamlit)

- Provides a user-friendly interface for prediction
- Accepts user input and sends it to the backend API
- Displays prediction results and system status

4.2 Backend (FastAPI)

- Exposes REST API endpoints
- Handles inference requests
- Loads trained model from artifacts

4.3 Machine Learning Model

- LightGBM-based binary classifier
- Predicts probability of loan default

4.4 Data Pipeline (DVC)

- Handles feature engineering, training, and evaluation
- Tracks data and model versions
- Maintains pipeline reproducibility

4.5 Workflow Orchestration (Airflow)

- Schedules and triggers data pipeline
- Detects new data and executes pipeline

4.6 Experiment Tracking (MLflow)

- Tracks model parameters, metrics, and artifacts
- Enables comparison of experiments

4.7 Monitoring (Prometheus & Grafana)

- Collects API metrics
- Visualizes system performance

5 System Workflow

1. User inputs data via Streamlit UI
2. UI sends request to FastAPI
3. FastAPI processes request and calls model
4. Model returns prediction
5. Result is displayed to the user
6. Prometheus collects metrics from API
7. Airflow triggers training pipeline when new data is available
8. DVC executes pipeline and updates model
9. MLflow logs experiments

6 Technology Stack

- Python (Core language)
- Streamlit (Frontend)
- FastAPI (Backend API)
- LightGBM (Model)
- DVC (Pipeline & versioning)
- MLflow (Experiment tracking)
- Airflow (Orchestration)
- Prometheus & Grafana (Monitoring)

7 Design Justification

- FastAPI provides high-performance APIs suitable for real-time inference
- DVC ensures reproducibility of ML workflows
- MLflow simplifies experiment tracking
- Airflow enables automated pipeline execution
- Prometheus allows real-time monitoring

8 Conclusion

The system is designed as a modular, scalable, and reproducible machine learning platform. The use of modern MLOps tools ensures robustness and ease of maintenance.